

General Information

- **Submission due date:** Due **@10:59 AM on 06-Oct-2026** (right before your next lab)
- **Lab weight:** 2%

OVERVIEW:

- Build a **Pizza Ordering System** form for a client
- Practice accessible **HTML tables**
- Practice accessible **HTML form elements**
- Perform client-side **HTML form validation**
- Apply modern **CSS layouts**

OBJECTIVES:

- **Semantic Form Structure:**
 - Use <fieldset>, <legend> and accessible <label> binding (for/id)
- **HTML5 Form Validation:**
 - Apply constraints such as required, pattern, minlength and min/max
- **Advanced CSS Layouts:**
 - Use CSS Grid for page structure and label/control alignment
 - Use Flexbox for the smaller groups (option groups and button row)
- **Accessibility:**
 - Create an accessible table using <caption>, <thead>, <tbody> and the <th> scope attribute
 - Create an accessible form with <label> elements linked via for/id
- **Responsiveness:**
 - Implement a mobile-first layout using media queries

PART 1: Analyze Reference Files

Review the following reference files to understand Grid/Flexbox nesting, validation styling, responsive layouts and other aspects that align with this lab. Highly recommended to run this locally.

HTML – *reference.html*

```
<!DOCTYPE html>
<html lang="en">

<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Reference Form</title>
  <link rel="stylesheet" href="reference.css">
</head>

<body>
  <main class="card">
    <h1>Reference Registration Form</h1>
    <p>Analyze this code to help you build the Pizza Ordering System.</p>

    <form action="#" method="post">
      <fieldset>
        <legend>Attendee Info (Grid Layout)</legend>
        <div class="form-grid">
          <label for="email">Email (Required):</label>
          <input type="email" id="email" required placeholder="test@test.com">

          <label for="web">Website (Pattern):</label>
          <input type="url" id="web" required pattern="https://.+"
            placeholder="Must start with https://">
        </div>
      </fieldset>

      <fieldset>
        <legend>Preferences (Flex Layout)</legend>
        <label>Dietary Restrictions:</label>
        <div class="flex-row">
          <label><input type="checkbox"> Gluten Free</label>
          <label><input type="checkbox"> Vegan</label>
          <label><input type="checkbox"> Halal</label>
        </div>
      </fieldset>

      <button type="submit">Register</button>
    </form>
  </main>
</body>
</html>
```

CSS – *reference.css*

```
body {
  font-family: sans-serif;
  background: #eee;
  padding: 20px;
}
```

```

.card {
  background: white;
  padding: 20px;
  max-width: 500px;
  margin: 0 auto;
}

fieldset {
  margin-bottom: 20px;
  border: 1px solid #ccc;
}

/* Grid example */
.form-grid {
  display: grid;
  grid-template-columns: 1fr 2fr;
  gap: 10px;
  align-items: center;
}

label {
  text-align: right;
  font-weight: bold;
}

/* Flex example */
.flex-row {
  display: flex;
  gap: 15px;
  margin-top: 5px;
}

/* Validation feedback */
input:invalid {
  border: 2px solid red;
}

input:valid {
  border: 2px solid green;
}

/* Responsive breakpoint */
@media (max-width: 600px) {
  .form-grid {
    grid-template-columns: 1fr;
  }

  label {
    text-align: left;
  }
}

```

PART 2: Setting Up and Constraints

Complete these steps before you start coding.

1. Create a folder: downtown-pizza
2. In VS Code: File -> Open Folder -> select downtown-pizza
3. Create files: **index.html** and **style.css**

Constraints:

1. **Language:** HTML and CSS only. No JavaScript is allowed.
2. **Accessibility:** Every form control must have a `<label>` linked via `for/id`. Related controls must be grouped in `<fieldset>` with a `<legend>`.
3. **Responsiveness:** The layout must adapt to mobile screens.

PART 3: Building the Pizza Ordering System

Downtown Pizzeria needs an online order form with strict data validation to reduce delivery errors. You will build this system for the client by adhering to the following HTML and CSS requirements.

1. HTML Requirements

Layout structure (critical):

- Use a `<main>` container that has two direct children (siblings):
 - A `<form>` containing all input sections. Form data will not be processed. Use `action="#"`.
 - An `<aside>` containing the Order Summary table.
 - Note: The table must NOT be placed inside the `<form>`. The `<form>` and `<aside>` must be siblings inside `<main>`.

Form content must include 3 sections wrapped in `<fieldset>` with a `<legend>`:

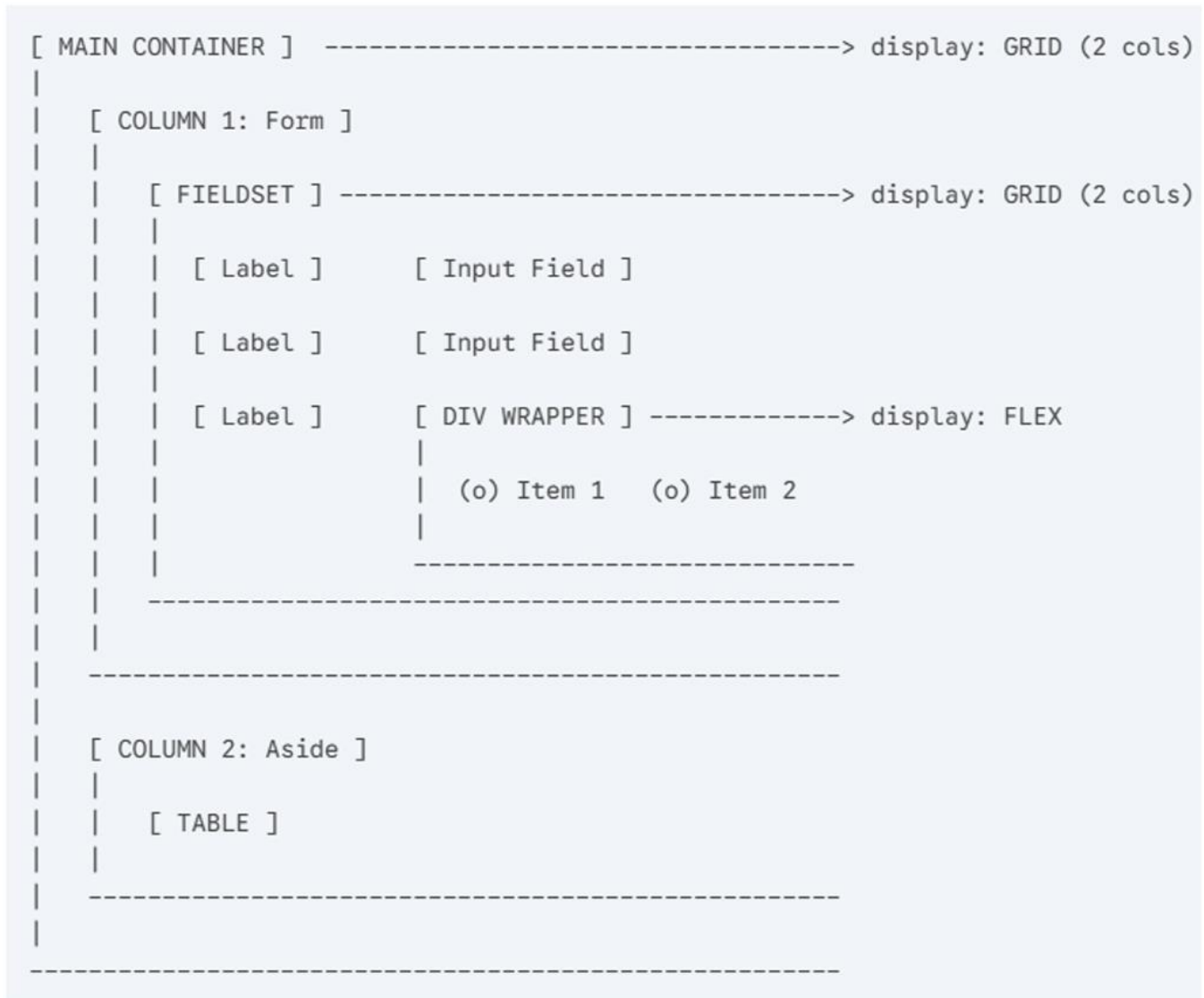
- Section 1 - Customer Information
 - Name: `<input type="text">` (Required, `minlength="2"`).
 - Address: `<input type="text">` (Required).
 - Phone: `<input type="tel">` (Required). Use `pattern="[0-9]{3}-[0-9]{3}-[0-9]{4}"`. Add placeholder: 123-456-7890.
- Section 2 - The Order
 - Crust: Radio buttons (Thin, Classic, Deep Dish). Required. Users must select exactly one.
 - Toppings: Checkboxes (at least 4 options, e.g., Pepperoni, Mushroom, Olives, Green peppers). No minimum-selection rule required.
 - Quantity: `<input type="number">` (Required). `min="1"`, `max="10"`.
- Section 3 - Delivery and Payment
 - Delivery instructions: `<textarea>` (Optional).
 - Preferred delivery date and time: `<input type="datetime-local">` (Required).
 - Payment method: `<select>` dropdown (Required) with options: Cash, Card, Debit.
 - Buttons: Include a Submit button (and an optional Reset button).

Order summary table (static example, no JavaScript needed):

- Inside the `<aside>` (the sibling to your form), create a `<table>` that displays a static example order summary.
- Your table must include: `<caption>`, `<thead>`, `<tbody>` and column headers (`<th>`) with `scope="col"`.
- Minimum 3 rows in `<tbody>` (e.g., Base pizza, Toppings, Delivery fee) plus a final Total row. You can use `<tfoot>` for the final Total row.

- The table does not need to update automatically so placeholder values will be used.
- You may copy these placeholder rows:
 - Base pizza | Large (12"), Thin crust | \$16.99
 - Toppings | Mushrooms, Olives, Extra cheese | \$4.50
 - Delivery fee | Standard delivery | \$3.99
 - Total | (example total) | \$25.48

2. CSS Requirements



Page layout (macro grid):

- Apply CSS Grid to the <main> container.
- Desktop: 2-column layout. The form should take roughly 66% (2fr) and the table 33% (1fr).
- Mobile: At a breakpoint (max-width: 600px), switch to 1 column so the table stacks below the form.

Form internals (micro grid):

- Inside each `<fieldset>`, use CSS Grid to align labels and controls in two columns.
- Use 150px for labels and 1fr for inputs (e.g., `grid-template-columns: 150px 1fr`).
- Right-align labels to visually connect them to their inputs, and vertically center items (`align-items: center`).
- Radio and Checkbox groups are an exception. Place their option list inside a wrapper element in the 1fr column and use Flexbox on that wrapper.

Option groups and Button row (Flexbox):

- Place radio, checkbox and button options list inside a wrapper element
- Crust (radios): Use Flexbox to lay these out in a single horizontal row.
- Toppings (checkboxes): Use a Flexbox that wraps (`flex-wrap: wrap`) so that options flow naturally even when viewed on narrow screens.
- Buttons: Use a Flexbox wrapper element to align the submit and reset buttons neatly.

Table styling:

- Collapse borders and add padding for readability.
- Add zebra striping using `:nth-child(even)` on `<tbody>` rows.
- Add a hover effect to `<tbody>` rows.
- Ensure the table remains usable on narrow screens (stacking is automatic via the macro grid; horizontal scrolling is acceptable if needed).

Visual feedback (validation):

- Style `input:valid` and `select:valid` with a green border.
- Style `input:invalid` and `select:invalid` with a red border.
- UX requirement: Do not show red borders on empty required fields as soon as the page loads. Only show red when the user has interacted.
- You may chain selectors with `:focus` or `:not(:placeholder-shown)` (for inputs with placeholders).
- Date must have a specified minimum value.
- **Note:** Example pattern: `input:not(:placeholder-shown):invalid { border: 2px solid red; }` (apply similar logic to other controls as needed).

Validation styling tip:

- When you style `:invalid` fields with a red border, browsers may mark required inputs as invalid immediately on page load—even before a user types anything. That creates a poor user experience.
- To avoid showing red borders too early, use this selector:
 - `input:not(:placeholder-shown):invalid { /* red border */ }`
- Meaning:
 - `:invalid` = the value does not meet the HTML rules (required, pattern, min/max, etc.)
 - `:placeholder-shown` = the placeholder text is still visible (usually the user has not typed yet)
 - `:not(:placeholder-shown)` = only apply the red border after the user starts typing

- Result: required fields are **not red on page load** but will turn red **after the user enters invalid data**.
- **Important:** This technique works only for inputs that have a placeholder. For fields like `<select>` or `datetime-local`, use a different approach such as `:focus:invalid` or another selector of your choice to show feedback **only after** the user interacts.

PART 4: Local Testing + Deployment

Before you deploy your website on GitHub, you must first confirm it works locally. Once local testing succeeds you will go ahead and deploy. You can follow these steps:

1. **Save your files**
 - Windows: **Ctrl + S**
 - Mac: **Cmd + S**
2. **Test Option A (Highly Recommended): Use Live Server in VS Code**
 - Right-click `index.html` → **Open with Live Server**
 - Your browser will open with a local address like <http://127.0.0.1:5500/>
 - Save changes and the page updates automatically (or refresh if needed)
3. **Test Option B (Simple): Open the file directly**
 - In VS Code, right-click `index.html`
 - Choose **Reveal in File Explorer** (Windows) / **Reveal in Finder** (Mac)
 - Double-click `index.html` to open it in your browser
4. **Verification checklist**
 - Layout: The page is 2 columns on desktop and 1 column on mobile (under 600px).
 - Structure: The table is NOT inside the `<form>` tag; it is a sibling inside `<main>`.
 - Validation: The phone field rejects values that do not match 123-456-7890 format.
 - UX: Empty required fields are not red when the page first loads.
 - Quantity: You cannot pick a number outside 1-10.
 - Accessibility: Clicking each label activates/focuses the correct control (for/id matches).
 - Accessibility: The table contains `<caption>`, `<thead>`, `<tbody>`, and `<th scope="col">` headers.
5. **Deploy to the Internet**
 - Refer to previous labs or use your preferred steps to **go live** on GitHub Pages.

SUBMISSION

Submit **one PDF document** through mēskanās containing your answers to the questions below:

1. Paste your **GitHub Repo URL** (e.g., <https://github.com/username/downtown-pizza>)
2. Paste your live **GitHub Pages URL** (e.g., <https://username.github.io/downtown-pizza>)
3. Can you explain how you made the form and table in this system **accessible**?
4. Why is HTML **form validation** important? What are two examples of validation rules used in this lab?
5. What is the difference between the **CSS Grid** and **Flexbox**? Can you provide an example of how each layout technique was used in this lab?

DEMO: Tuesday, October 6, 2026 @11:00 AM

- Demonstrate your completed lab during the lab session.
- Be prepared to explain your solution and answer questions about your code.

MARKING SCHEME

Component	Mark
Lab Questions (PDF Submission)	5
Lab Demo	15
Total	20